
Departamento de Engenharia Informática

Play4Change

Aplicação de Aprendizagem Adaptativa para Educação Cívica

Dossier de Projecto — Documento para o Júri

Relatório para a Unidade Curricular de Projeto e Seminário
da Licenciatura em Engenharia Informática e de Computadores

Aluno: A51602 — Radesh Ilesh Gamanbhai Govind

Email: A51620@alunos.isel.pt

Professor Nuno Datia

Orientadores: Professor António Serrador
Professor Michel Vorenhout

Contents

1	Descrição do Projecto	2
1.1	Contexto e Motivação	2
1.2	Arquitectura do Sistema	2
1.3	Stack Tecnológica	3
2	Repositórios e Acesso	3
2.0.1	Tipo de Acesso	3
3	Requisitos do Sistema	4
3.1	Pré-requisitos Obrigatórios	4
3.2	Pré-requisitos Opcionais (apenas para o painel web)	4
4	Variáveis de Ambiente e Credenciais	4
4.1	Ficheiro <code>.env</code> (backend / Docker Compose)	5
4.2	Credenciais de Acesso aos Serviços	5
4.3	Comportamento sem Chaves Externas	5
5	Instalação e Arranque	6
5.1	Passo 1 — Clonar o Repositório Principal	6
5.2	Passo 2 — Verificar o Ficheiro <code>.env</code>	6
5.3	Passo 3 — Arrancar todos os Serviços	6
5.4	Passo 4 — Confirmar que os Serviços estão Activos	6
5.5	Passo 5 — Parar o Sistema	7
6	Scripts de Apoio	7
7	Guia de Verificação para o Júri	9
7.1	Verificação 1 — Health Check da API	9
7.2	Verificação 2 — Documentação Swagger da API	9
7.3	Verificação 3 — Criar uma Conta (Magic Link)	9
7.4	Verificação 4 — Promover Conta a Administrador	10
7.5	Verificação 5 — Estatísticas Públicas da Plataforma	10
7.6	Verificação 6 — Criar um Tópico de Aprendizagem via IA	10
7.7	Verificação 7 — Inscrição num Tópico e Tarefa Diária	11
7.8	Verificação 8 — Observabilidade (Grafana + Prometheus)	12
7.9	Verificação 9 — MinIO Console (Armazenamento)	12
8	Painel de Administração Web	12
9	Resumo Rápido para o Júri	13

1. Descrição do Projecto

1.1. Contexto e Motivação

A transição para cidades inteligentes enfrenta três obstáculos interdependentes: infraestruturas tecnológicas, constrangimentos financeiros e de governança, e — o foco deste trabalho — **o elemento humano**: as competências, a motivação e a participação dos cidadãos. A Aliança Universitária **U!REKA** tem como missão acelerar precisamente esta transição.

O *Play4Change* propõe uma plataforma de **aprendizagem adaptativa e gamificada para educação cívica urbana**, desenvolvida no âmbito da missão U!REKA e alinhada com os Objetivos de Desenvolvimento Sustentável das Nações Unidas (ODS 4, 11 e 13). A plataforma assenta em três pilares:

1. **Engagement — Desafios diários gamificados**: transformam a aprendizagem cívica num hábito diário motivante, através do qual os cidadãos desenvolvem micro-competências progressivas;
2. **Personalização — Desafios adaptativos por IA**: um agente de inteligência artificial (Mistral AI) adapta os percursos ao perfil e às dificuldades de cada cidadão. Quando gera um percurso alternativo, este é indexado em *pgvector* e reutilizado automaticamente para perfis análogos, reduzindo a dependência de inferência em tempo real;
3. **Reconhecimento — Badges e micro-competências**: cada etapa de progressão é certificada por um *badge*, tornando cada passo no percurso visível e contabilizável.

1.2. Arquitectura do Sistema

O sistema foi concebido segundo os princípios de **Arquitetura Hexagonal** (Ports & Adapters), em cinco contextos delimitados (DDD):

Contexto	Responsabilidade
<i>Identity</i>	Registo, autenticação (magic link), gestão de perfil
<i>Topic</i>	Criação de tópicos via URL/PDF, geração de tarefas por IA (pipeline assíncrono)
<i>Enrollment</i>	Inscrição, entrega diária de tarefas, progressão por dia com timezone
<i>Struggle</i>	Sessões de remediação adaptativa quando o utilizador falha

1.3. Stack Tecnológica

Componente	Tecnologia
Backend	Kotlin 2.x + Spring Boot 3.2 (JDK 21)
Frontend Web	React 18 + TypeScript 5.7 + Vite + Tailwind CSS 4
Base de Dados	PostgreSQL 16 + extensão <code>pgvector</code> (busca semântica)
Cache	Redis 7
Armazenamento	MinIO (S3-compatible, para PDFs e conteúdo ingerido)
IA / LLM	Mistral AI <code>mistral-small-latest</code> via LangChain4j
Autenticação	JWT + Magic Link (Resend)
Migrações DB	Flyway (V1–V23, 23 migrações)
Observabilidade	Micrometer + Prometheus + Grafana
Orquestração	Docker Compose (7 serviços)

2. Repositórios e Acesso

O projecto está distribuído por três repositórios na organização GitHub **UREKA-Play4Change**:

Repositório	Conteúdo
<code>play4change-core</code> github.com/UREKA-Play4Change/play4change-core	Backend Spring Boot (Kotlin), AI agent LangChain4j, infra Docker Compose (7 serviços), cliente Compose multiplatform
<code>play4change-web</code> github.com/UREKA-Play4Change/play4change-web	Painel de administração web (React 18 + TypeScript + Vite + Tailwind CSS)
<code>play4chage-report</code> github.com/UREKA-Play4Change/play4chage-report	Relatório académico (LaTeX)

2.0.1. Tipo de Acesso

Os repositórios são públicos. Não são necessárias credenciais para clonar ou consultar o código. O júri pode aceder directamente sem criar conta GitHub.

Para clonar todos os repositórios:

```
# Backend + infra (repositório principal)
git clone
  ↪ https://github.com/UREKA-Play4Change/play4change-core.git

# Painel de administração web
git clone
  ↪ https://github.com/UREKA-Play4Change/play4change-web.git

# Relatório (opcional)
git clone
  ↪ https://github.com/UREKA-Play4Change/play4change-report.git
```

3. Requisitos do Sistema

3.1. Pré-requisitos Obrigatórios

Para correr o sistema completo basta ter instalado:

Ferramenta	Versão mínima	Verificação
Docker Engine	24.x	<code>docker --version</code>
Docker Compose	v2.x (plugin)	<code>docker compose version</code>
Git	qualquer	<code>git --version</code>

Nota: O Docker Compose v2 utiliza `docker compose` (sem hífen). Se o sistema usar a versão v1 antiga, o comando é `docker-compose`. Este documento usa a sintaxe v2.

3.2. Pré-requisitos Opcionais (apenas para o painel web)

Necessário apenas se o júri quiser correr o frontend em modo de desenvolvimento (a instalação Docker já inclui a versão de produção compilada):

Ferramenta	Versão mínima	Verificação
Node.js	20.x LTS	<code>node --version</code>
npm	10.x	<code>npm --version</code>

4. Variáveis de Ambiente e Credenciais

4.1. Ficheiro .env (backend / Docker Compose)

O repositório `play4change-core` inclui um ficheiro `.env` na raiz com todos os valores necessários para execução local. **O júri não precisa de criar nem editar este ficheiro** — está pronto a usar.

As variáveis críticas e os seus valores de demonstração são:

Variável	Valor demo	Propósito
<code>DB_USER</code>	<code>play4change</code>	Utilizador PostgreSQL
<code>DB_PASS</code>	<code>play4change</code>	Password PostgreSQL
<code>JWT_SECRET</code>	<i>(no .env)</i>	Chave de assinatura JWT
<code>MISTRAL_API_KEY</code>	<i>(no .env)</i>	Chave Mistral AI
<code>RESEND_API_KEY</code>	<i>(no .env)</i>	Chave email (Resend)
<code>MINIO_ROOT_USER</code>	<code>minioadmin</code>	Utilizador MinIO
<code>MINIO_ROOT_PASSWORD</code>	<code>minioadmin</code>	Password MinIO
<code>GRAFANA_ADMIN_PASSWORD</code>	<code>admin</code>	Password Grafana

4.2. Credenciais de Acesso aos Serviços

Credenciais de demonstração (locais):

Serviço	URL	Utilizador	Password
PostgreSQL	<code>localhost:5432</code>	<code>play4change</code>	<code>play4change</code>
MinIO Console	<code>http://localhost:9001</code>	<code>minioadmin</code>	<code>minioadmin</code>
Grafana	<code>http://localhost:3000</code>	<code>admin</code>	<code>admin</code>

4.3. Comportamento sem Chaves Externas

O sistema foi desenhado para funcionar em modo degradado quando as chaves externas não estão disponíveis:

- **Sem `MISTRAL_API_KEY`:** a geração de tarefas usa um adaptador stub (`NoOpTaskGenerationAdapter`) — os tópicos são criados mas as tarefas ficam vazias. Para ver IA em acção, a chave deve estar configurada no `.env`.
- **Sem `RESEND_API_KEY`:** os e-mails de magic link são impressos nos **logs do servidor** em vez de serem enviados. O júri pode obter o link directamente nos logs com:

```
docker compose logs server | grep "magic-link"
```

5. Instalação e Arranque

5.1. Passo 1 — Clonar o Repositório Principal

```
git clone
  ↪ https://github.com/UREKA-Play4Change/play4change-core.git
cd play4change-core
```

5.2. Passo 2 — Verificar o Ficheiro .env

O ficheiro .env já existe na raiz do repositório. Verificar que está presente:

```
ls -la .env
cat .env    # confirmar que os valores estao preenchidos
```

5.3. Passo 3 — Arrancar todos os Serviços

```
docker compose up --build
```

Na primeira execução, o Docker irá:

1. Descarregar as imagens base (pgvector, MinIO, Redis, Nginx, Prometheus, Grafana);
2. Compilar o servidor Spring Boot (demora 2–4 minutos);
3. Executar as 23 migrações Flyway automaticamente;
4. Iniciar todos os serviços com health checks.

O sistema está pronto quando se vir no terminal:

```
play4change-server | Started Application in X.XXX seconds
```

Para correr em background:

```
docker compose up --build -d
```

5.4. Passo 4 — Confirmar que os Serviços estão Activos

```
docker compose ps
# Todos os servicos devem estar em estado "running" ou
  ↪ "healthy"
```

Serviço	Porta	Estado esperado
postgres	5432	healthy
redis	6379	healthy
minio	9000, 9001	healthy
server	8080, 9090	healthy
nginx	80	running
prometheus	9090	running
grafana	3000	running

5.5. Passo 5 — Parar o Sistema

```
# Parar sem apagar dados
docker compose down

# Parar e apagar todos os dados (reset completo)
docker compose down -v
```

6. Scripts de Apoio

O repositório `play4change-core` inclui um directório `scripts/` com utilitários para simplificar operações comuns. Conceder permissão de execução na primeira utilização:

```
chmod +x scripts/*.sh
```


Script	Descrição e exemplo de uso
<code>scripts/check-health.sh</code>	Aguarda que todos os serviços (postgres, redis, minio, server) estejam prontos e imprime o estado de cada um. Aceita <code>--timeout 120</code> para ajustar o tempo de espera. <code>./scripts/check-health.sh</code>
<code>scripts/logs.sh</code>	Segue os logs do Docker Compose com filtro opcional por serviço e padrão de texto. <code>./scripts/logs.sh server</code> <code>./scripts/logs.sh server magic-link</code>
<code>scripts/promote-admin.sh</code>	Promove um utilizador a ADMIN pelo e-mail (o utilizador deve ter feito login pelo menos uma vez). <code>./scripts/promote-admin.sh</code> <code>jury@example.com</code>
<code>scripts/setup.sh</code>	Apaga todos os volumes, reconstrói e arranca todos os serviços de raiz. Atenção: destrói dados existentes. <code>./scripts/setup.sh</code>
<code>scripts/restart-server.sh</code>	Recompila o JAR do servidor e reinicia apenas o contentor server , preservando os dados de postgres, minio e redis. <code>./scripts/restart-server.sh</code>
<code>scripts/db-shell.sh</code>	Abre uma sessão interactiva <code>psql</code> no contentor postgres. <code>./scripts/db-shell.sh</code>
<code>scripts/db-query.sh</code>	Executa uma instrução SQL directamente no postgres. <code>./scripts/db-query.sh "SELECT id, email, role FROM users;"</code>
<code>scripts/migration-status.sh</code>	Mostra o historial completo de migrações Flyway: versão, descrição, estado e data de instalação. <code>./scripts/migration-status.sh</code>
<code>scripts/minio-init.sh</code>	Cria o bucket MinIO necessário. Requer o cliente mc instalado (<code>brew install minio/stable/mc</code>). <code>./scripts/minio-init.sh</code>
<code>scripts/run-tests.sh</code>	Corre os testes com o perfil correcto (server , ai ou all). <code>./scripts/run-tests.sh server</code> <code>./scripts/run-tests.sh all</code>
<code>scripts/build-android.sh</code>	Compila o APK Android em modo debug (por defeito) ou release. <code>./scripts/build-android.sh</code>

7. Guia de Verificação para o Júri

Esta secção descreve o percurso completo que o júri pode seguir para verificar que o sistema funciona correctamente, da autenticação à entrega adaptativa de tarefas.

7.1. Verificação 1 — Health Check da API

```
curl http://localhost:9090/actuator/health
# Resposta esperada:
# {"status": "UP", "components": {"db": {"status": "UP"}, ...}}
```

Ou abrir no browser: <http://localhost:9090/actuator/health>

7.2. Verificação 2 — Documentação Swagger da API

Abrir no browser:

<http://localhost:8080/swagger-ui.html>

Todos os endpoints estão documentados e podem ser testados directamente na interface Swagger.

7.3. Verificação 3 — Criar uma Conta (Magic Link)

1. Aceder ao painel de administração web em: <http://localhost:80>
2. No ecrã de login, introduzir um endereço de e-mail (qualquer e-mail, real ou fictício).
3. Clicar em “**Enviar Magic Link**”.
4. Como o ambiente é local sem chave Resend, obter o link nos logs do servidor:

```
docker compose logs server | grep -i
↪ "magic\|verify\|token"
```

5. Copiar o URL de verificação do log e abrir no browser. A sessão é autenticada automaticamente.

Alternativa via API directamente (sem frontend):

```
# 1. Solicitar magic link
curl -X POST http://localhost:8080/auth/magic-link \
  -H "Content-Type: application/json" \
  -d '{"email": "jury@example.com"}'
```

```
# 2. Ver o token nos logs
docker compose logs server | grep "verify"

# 3. Verificar o token (substituir TOKEN pelo valor obtido)
curl "http://localhost:8080/auth/verify?token=TOKEN"
# Resposta: {"accessToken":"...", "refreshToken":"..."}
```

7.4. Verificação 4 — Promover Conta a Administrador

Após criar a conta com o e-mail `jury@example.com` (ou o e-mail escolhido), promover a administrador:

```
# Na raiz do repositório play4change-core
chmod +x scripts/promote-admin.sh
./scripts/promote-admin.sh jury@example.com
```

Saída esperada:

```
OK: 'jury@example.com' promoted to ADMIN.
They must log out and log in again for the new role to take
  ↪ effect.
```

Após a promoção, fazer logout e novo login para o JWT reflectir o papel de ADMIN.

7.5. Verificação 5 — Estatísticas Públicas da Plataforma

```
curl http://localhost:8080/api/stats/public
# Resposta:
  ↪ {"totalUsers":1, "totalTopics":0, "totalEnrollments":0, ...}
```

7.6. Verificação 6 — Criar um Tópico de Aprendizagem via IA

Este passo requer a chave `MISTRAL_API_KEY` configurada no `.env`. Com a conta ADMIN autenticada:

Via Painel Web (recomendado):

1. Navegar para `http://localhost/admin/topics/new`
2. Introduzir um URL de conteúdo cívico (ex.: página Wikipedia sobre reciclagem)

urbana)

3. Seleccionar dificuldade, idioma e categoria
4. Clicar em “**Criar Tópico**”
5. Acompanhar o progresso em tempo real (SSE) na página de detalhes do tópico

Via API (com token JWT de ADMIN):

```
curl -X POST http://localhost:8080/admin/topics \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ACCESS_TOKEN" \
  -d '{
    "sourceUrl":
      ↪ "https://pt.wikipedia.org/wiki/Reciclagem",
    "language": "pt",
    "durationDays": 7,
    "difficulty": "BEGINNER",
    "category": "ENVIRONMENT"
  }'
```

Acompanhar progresso: [GET /admin/topics/{id}/progress](#) (SSE stream)

7.7. Verificação 7 — Inscrição num Tópico e Tarefa Diária

Com a conta (papel USER) e um tópico activo (`status: ACTIVE`):

```
# Inscrever no topico (substituir TOPIC_ID e ACCESS_TOKEN)
curl -X POST http://localhost:8080/topics/TOPIC_ID/enroll \
  -H "Authorization: Bearer ACCESS_TOKEN"

# Obter a tarefa do dia
curl http://localhost:8080/tasks/today?topicId=TOPIC_ID \
  -H "Authorization: Bearer ACCESS_TOKEN"

# Submeter resposta (substituir ASSIGNMENT_ID e escolha
  ↪ A/B/C/D)
curl -X POST http://localhost:8080/tasks/ASSIGNMENT_ID/submit \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ACCESS_TOKEN" \
  -d '{"selectedOption": "A"}'
```

7.8. Verificação 8 — Observabilidade (Grafana + Prometheus)

- **Grafana:** <http://localhost:3000> — login: `admin / admin`
Dashboards pré-provisionados mostram métricas de geração de tarefas, latências P50/P95/P99, e contadores de submissões.
- **Prometheus:** <http://localhost:9090>
Métricas brutas em `/actuator/prometheus` (porta 9090 do servidor).

7.9. Verificação 9 — MinIO Console (Armazenamento)

- Aceder a <http://localhost:9001>
- Login: `minioadmin / minioadmin`
- O bucket `play4change` contém o conteúdo ingerido dos tópicos (PDF ou HTML extraído)

8. Painel de Administração Web

O painel web (`play4change-web`) está incluído na imagem Docker do Nginx em modo de produção. Não é necessário correr o servidor de desenvolvimento Node.js para aceder ao painel.

URL	Funcionalidade
http://localhost/	Página pública com missão e estatísticas em tempo real
http://localhost/admin/login	Login (magic link)
http://localhost/admin/dashboard	Dashboard com resumo de actividade
http://localhost/admin/topics	Listagem de todos os tópicos (com filtros de estado)
http://localhost/admin/topics/new	Criar tópico a partir de URL ou PDF
http://localhost/admin/topics/{id}	Detalhe, edição de tarefas, pré-requisitos
http://localhost/admin/learning-paths	Visualização do grafo de pré-requisitos (DAG)
http://localhost/admin/users	Gestão de utilizadores e promoção a ADMIN
http://localhost/admin/reports	Relatórios de tarefas sinalizadas pelos utilizadores
http://localhost/admin/badges	Badges de micro-competências por tópico

Nota sobre autenticação: O painel web requer conta com papel **ADMIN**. A primeira conta criada via magic link tem papel **USER** por defeito. Usar o script `./scripts/promote-admin.sh <email>` para promover a ADMIN (ver Secção 7.4).

9. Resumo Rápido para o Júri

Checklist mínima para verificar o projecto:

1. **Clonar:** `git clone https://github.com/UREKA-Play4Change/play4change-core.git`
&& `cd play4change-core`
2. **Arrancar:** `docker compose up --build` (aguardar “Started Application”)
3. **Health check:** `curl http://localhost:9090/actuator/health`
4. **Swagger:** `http://localhost:8080/swagger-ui.html`
5. **Painel web:** `http://localhost` → Login → Magic Link → obter token nos logs
6. **Promover a ADMIN:** `./scripts/promote-admin.sh <email>` (re-login

necessário)

7. **Grafana:** <http://localhost:3000> (admin/admin)

8. **MinIO:** <http://localhost:9001> (minioadmin/minioadmin)

Junho 2026 · ISEL — Instituto Superior de Engenharia de Lisboa · U!REKA European
University

Radesh Ilesh Gamanbhai Govind (A51602) · A51620@alunos.isel.pt